Teaching KelanaAI to Remember Conversations

Build KelanaAI with Python, Next.js & Amazon Bedrock — Session 10

Duration: 2 Hours

Week 4 · Conversation Memory

Feature: Conversational AI Chat

Yesterday KelanaAI learned how to read trusted knowledge.

Today KelanaAI learns how to remember what users said earlier.

SFIA Competency Card

Session 10 — Teaching KelanaAI to Remember Conversations

FEATURE DELIVERED

✓ **Conversational AI Chat**

PRIMARY SFIA SKILLS

INAI L3	Artificial Intelligence Implementation
SWDN L3	Software Design
PROG L2	Programming / Software Development
DATM L2	Data Management

EVIDENCE — BY END OF SESSION, STUDENTS PRODUCE:

- ✓ Chat interface
- ✓ Conversation database
- ✓ Context-aware AI responses
- ✓ Multi-turn conversation support
- ✓ Git commit

✓ capstone-ready

Learning Objectives

By the end of this session, students will be able to:

01

Explain why LLMs are stateless.

Articulate the misconception and the real architecture.

02

Design a conversation history model.

Two tables: conversations and messages.

03

Store chat history in PostgreSQL.

Persist every turn with role + content + timestamp.

04

Build prompts using previous messages.

Reconstruct context before calling Bedrock.

05

Generate context-aware AI responses.

Day-2 follow-up questions just work.

06

Display conversations in the frontend.

Render a chat UI in Next.js with message history.

Product Roadmap

Where we are — Week 4, Conversation Memory

You are here

WEEK 1	WEEK 2	WEEK 3	WEEK 4 · TODAY
✓ Trip Summary	✓ PostgreSQL	✓ Trip Dashboard	▶ Conversation Memory
✓ Recommendation Engine	✓ Amazon Bedrock	✓ Authentication	✓ Deployment
✓ REST API	✓ Next.js Frontend	✓ RAG	✓ Capstone

→ Yesterday: KelanaAI read trusted knowledge (RAG). **Today: KelanaAI remembers what users said.**

Story — Why Memory Matters

Imagine the following conversation.

USER
Plan a family trip to Japan.

AI
Great! Here's a 5-day itinerary starting in Tokyo...

USER
What should we do on Day 2?

?

How does the AI know the user is talking about Japan?
It doesn't — unless the application provides the previous conversation.

Today's goal: build that memory.

Technical Lesson

Eight parts — from stateless LLMs to context-window trimming.

01 Stateless LLMs

02 DB Design

03 Conv. APIs

04 Send Message

05 Prompt Builder

06 Chat UI

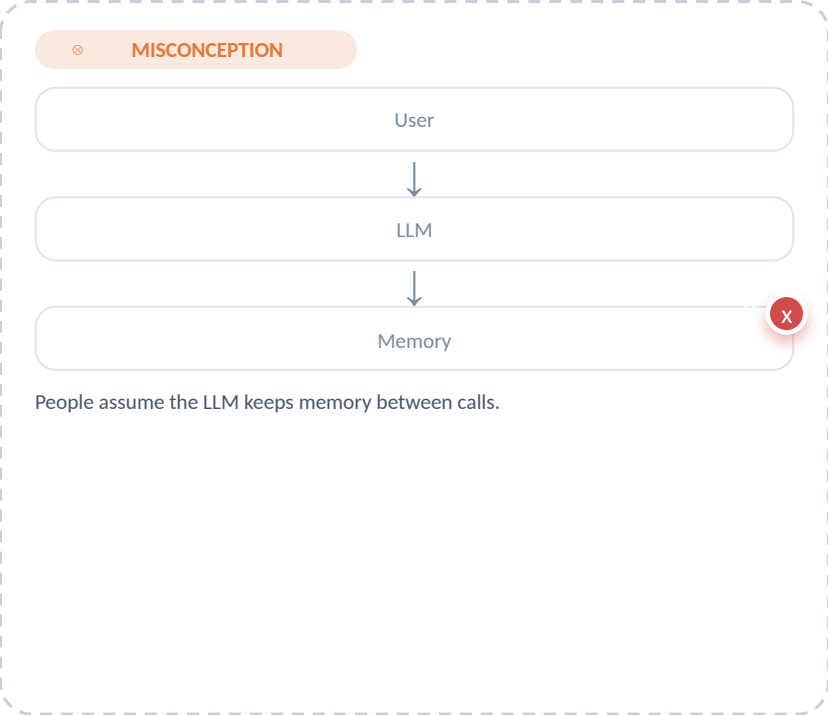
07 Continue Conv.

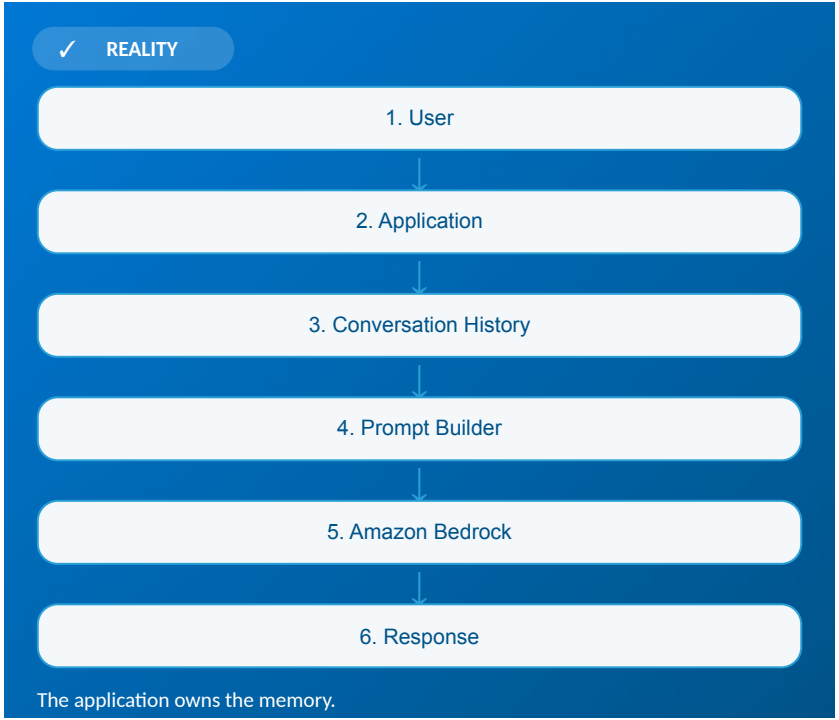
08 Trim Context

The application owns the memory.

Part 1 — LLMs Are Stateless

Many people believe the model remembers. It does not.

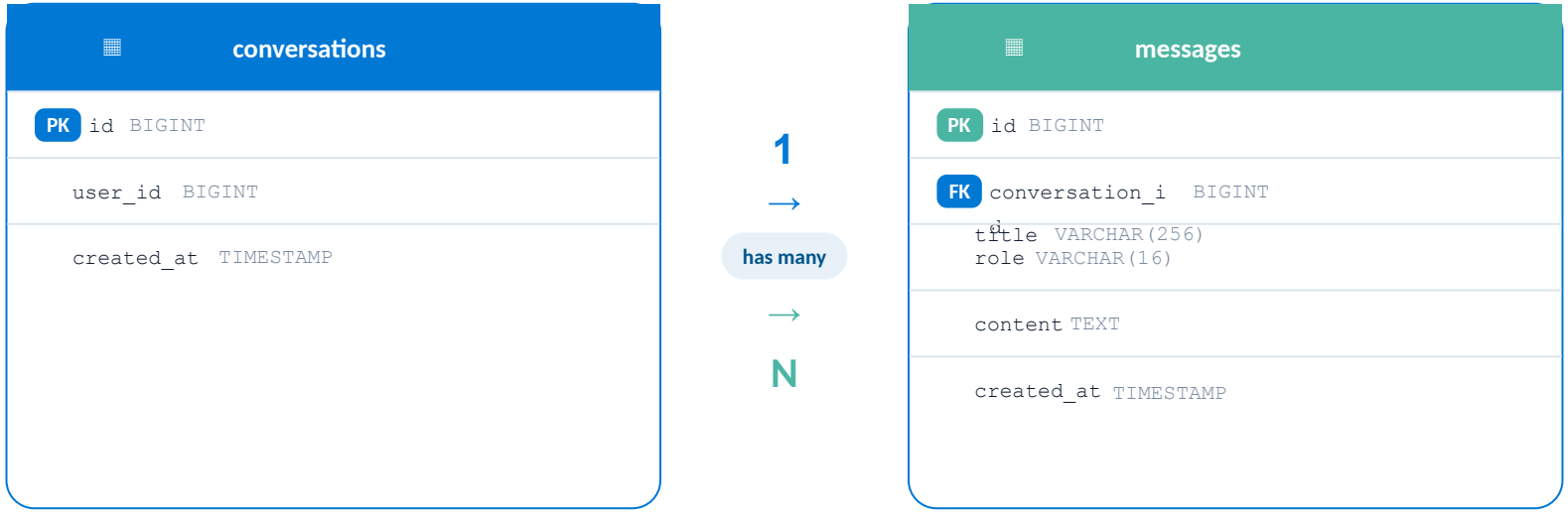




Takeaway: memory is an application-layer concern, not a model feature.

Part 2 — Database Design

Two tables model the entire conversation history.



Relationship: one conversation → many messages.

Every conversation contains multiple messages, ordered by `created_at`.

Part 3 — Conversation APIs

Two endpoints to start and list conversations.

POST `/api/v1/conversations` — CREATE

Create a new conversation row and return its identifier.

RESPONSE
201

```
{
  "conversation_id": 1
}
```

GET `/api/v1/conversations` — LIST

List previous conversations for the authenticated user.

id	title	created_at
1	Japan Family Trip	2025-07-02 14:22
2	Singapore Food Tour	2025-07-01 09:10

✦ These endpoints are thin — the heavy lifting happens in the message endpoint.

Part 4 — Send Message API

`POST /api/v1/conversations/{id}/messages` — the orchestration lives in our backend, not the model.

ENDPOINT

POST

`/api/v1/conversations/{id}/messages`

KEY PRINCIPLE

This orchestration is handled by our backend — not by the model.

01 Receive User Message

02 Save Message

03 Load Previous Messages

04 Build Prompt

05 Amazon Bedrock

06 Save AI Response

07 Return Response

Part 5 — Prompt Builder

Construct prompts from conversation history — a key responsibility of AI-native applications.

NAIVE **Without Context**

```
// Send only the latest message
prompt = "What should we do on Day 2?"
```

The LLM has no idea this is about Japan.

CONTEXT-AWARE **With Conversation History**

```
// Reconstruct the full conversation
prompt = [
  {role: 'user',      content: 'Plan a family trip to Japan.'},
  {role: 'assistant', content: 'Here is a 5-day itinerary…'},
  {role: 'user',      content: 'What should we do on Day 2?'}
]
```

The LLM now sees the full thread and answers correctly.

Prompt construction is a key responsibility of AI-native applications.

Part 6 — Next.js Chat Interface

The `/chat` route — frontend displays messages, backend manages context.

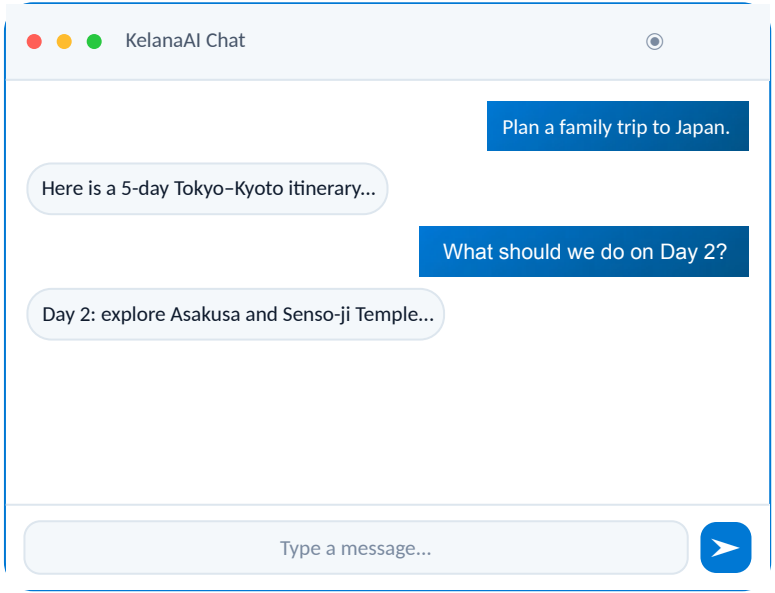
Route: /chat

A single-page chat experience inside the Next.js app.

Two responsibilities

Frontend: render messages, capture input. Backend: store + retrieve history, call Bedrock.

The frontend simply displays messages. The backend manages conversation context.



Part 7 — Continue Existing Conversations

Reload previous messages before calling Amazon Bedrock.

01

Conversation History

User clicks a past conversation.

→

02

Reload Messages

App fetches every previous message from DB.

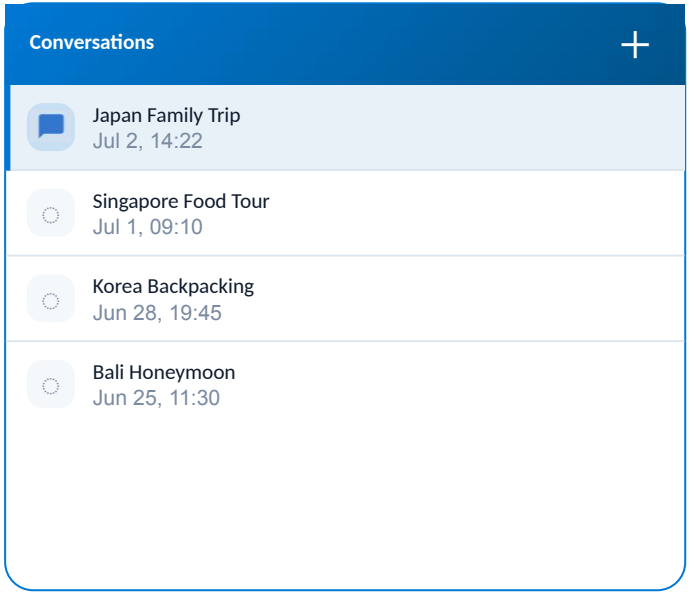
→

03

Continue Chatting

New prompt = full history + new message.

The application reloads previous messages before calling Amazon Bedrock.



Same Send Message endpoint — only the `conversation_id` differs.

Part 8 — Trim Context Window

What happens after 500 messages? Should we send all of them every time?

500+

MESSAGES IN A LONG THREAD

~8K

TOKENS PER LONG HISTORY

$$

COST GROWS WITH EVERY CALL

DISCUSSION — INTRODUCE THE IDEA OF:

Recent Messages

Send only the last N turns — cheaper, usually enough context.

Summarization

Compress older turns into a summary message.

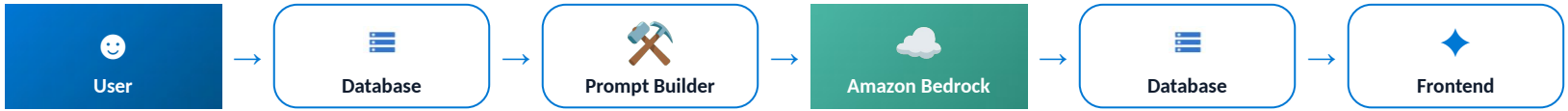
Token Limits

Every model has a max context window — never exceed it.

We won't implement advanced optimization today — but be aware of the tradeoffs.

Hands-on Lab

Build: POST /api/v1/conversations/{id}/messages — end-to-end orchestration.



✓ **Expected experience** User sends a Japan trip prompt → AI responds → user asks a Day 2 follow-up → AI understands the previous context.

TURN 1

Plan a Japan trip.

Here is a 5-day Tokyo–Kyoto itinerary with family-friendly stops...

TURN 2 — CONTEXT-AWARE

What about Day 2?

Day 2: explore Asakusa, Senso-ji Temple, and the Sumida River cruise.

Challenge

Extend today's feature with two UX wins.

CORE CHALLENGE

Conversation List on the Left

Display a conversation list in a left sidebar. Clicking a conversation loads all its previous messages into the chat panel.

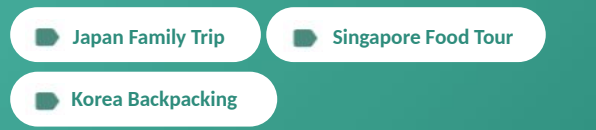
ACCEPTANCE CRITERIA

- ✓ Sidebar lists every conversation for the current user
- ✓ Click loads messages into the existing chat view
- ✓ New conversations appear in the list automatically

BONUS

Rename Conversations

Allow users to rename conversations to something meaningful.




```
Hint: PATCH /api/v1/conversations/{id}
```

Git Checkpoint

Commit today's work — KelanaAI now remembers conversations.

bash — kelana-ai

```
# stage today's work
git add .
# commit with a descriptive message
git commit -m "Add conversational memory"
# push to remote
git push
```

✦ Today KelanaAI learned how to remember conversations.

Tag your commit so it's easy to find later: git tag session-10

Mini Quiz

Quick check — discuss with your neighbour, then we'll review.

Q1

Why are LLMs considered stateless?

Hint: Think about what happens between two API calls.

Q2

Who owns conversation memory?

Hint: It's not the model.

Q3

Why do we rebuild prompts using previous messages?

Hint: The model has no built-in history.

Q4

Why shouldn't every previous message always be sent to the LLM?

Hint: Think cost, latency, and token limits.

Discuss for 5 minutes. We'll review answers together.

Homework & What's Next

Polish the chat experience tonight — and preview tomorrow.

HOMEWORK — IMPROVE THE CHAT EXPERIENCE

Add these four UX wins to your chat interface:

01 ✦ Conversation title

02 ↓ Auto-scroll to latest message

03 ··· Typing indicator

04 ◔ Timestamp for each message

↑ **Commit and push your solution.**

NEXT SESSION — 11

Deploy KelanaAI to the Cloud

Today KelanaAI became a true conversational AI assistant. Users can ask follow-up questions naturally without repeating previous information.

"However... only developers running the project locally can use it."

Tomorrow: deploy to the cloud — accessible from anywhere with a web browser.

MILESTONE ACHIEVED

KelanaAI is now a Conversational AI Assistant

You implemented one of the most important concepts in AI Native Engineering.

LLMs do not remember conversations.
Models are stateless by design.

Applications manage history.
Memory lives in your DB, not the model.

Prompt builders reconstruct context.
Every call rebuilds the full thread.

Foundation models reason over context.
Given context, they answer brilliantly.

KelanaAI is feature-complete — ready for its final step: deployment to the cloud.